Running an AI Agent in Production

What Nobody Tells You About the Economics of Always-On AI

Eliran Keren | Deeplica | March 2026

The $0.63 Hi

We deployed Eva — a personal AI assistant built on OpenClaw — as an experiment to understand what it actually takes to operate an AI agent in the real world. Not in a demo. Not in a sandbox. In production, connected to real messaging channels, handling real conversations.

Within two weeks, our API bill exploded. A single "hi" message was sending 169,000 tokens to the model. Every request cost $0.63. For a personal assistant that's supposed to save time and money, this was the opposite of the thesis.

Most people experience AI through chat interfaces — you type, it responds, you pay for what you use. Running an AI agent in production is something entirely different. The agent is always on. It processes heartbeats, monitors group chats, loads context, maintains sessions. The economics are not the economics of conversation. They're the economics of an always-on system.

This is what we learned.

What We Thought We Were Building

The setup was straightforward. Eva ran inside a Docker container on macOS, connected to Telegram for direct messages and a team broadcasting group, and to WhatsApp for personal and group conversations. She used Claude Sonnet 4.5 as her primary model, with Anthropic's prompt caching enabled to keep costs reasonable.

The architecture was sound — Docker isolation, read-only filesystem, non-root user, manual confirmation for sensitive actions. We had followed the security playbook.

What we hadn't done was follow the economics playbook. Because for most people building agents, that playbook doesn't exist yet.

What Actually Happened

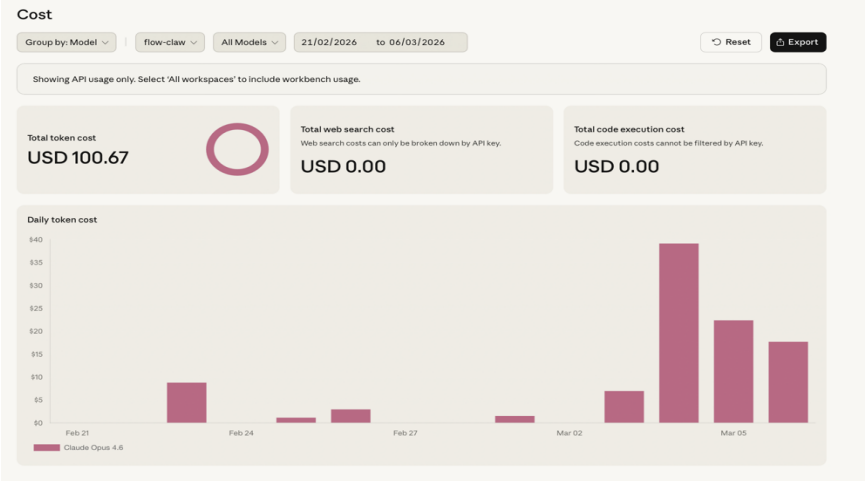
Three signals appeared almost simultaneously:

- The bill was climbing fast. What should have been a modest personal assistant cost was heading toward hundreds of dollars in the first two weeks.

- Rate limits kept hitting. Anthropic's usage limits were being reached repeatedly. Eva would stop responding mid-conversation — not because she couldn't answer, but because the API was throttled.

- The logs revealed the real problem. Every request — even a one-word message — was consuming 169,000 input tokens. That's 85% of Claude's 200K context window, used up before Eva even started thinking about a response.

The cost wasn't coming from Eva being verbose. It was coming from the sheer volume of context the system was loading for every single interaction.



API logs showing consistent 169K input tokens across every request

Pulling the Thread

WhatsApp Was a Wide-Open Firehose

WhatsApp's DM policy wasn't set to "disabled." Group policy wasn't set to "allowlist." This meant Eva was silently processing messages from WhatsApp groups she had no business being in. Each message triggered a full API call with the entire context window attached. This single misconfiguration accounted for an estimated 70% of the bill.

The Cache Was a Mirage

Anthropic's prompt caching is supposed to reduce costs by reusing the system prompt across requests. Ours was invalidating constantly:

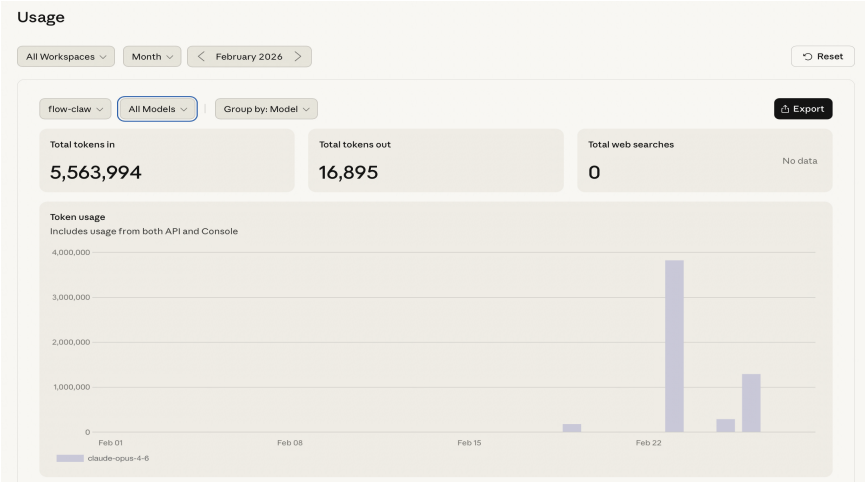
- Dynamic metadata: OpenClaw injects timestamps and runtime info into the system prompt. One changed byte invalidates the entire cache.
- Short TTL: The cache TTL was set to 5 minutes. Every interaction was a cold start.
- Tool call multiplication: Each tool call resends the full context. One message triggering 5 tool calls means 5x the context sent.

48 Zombie Sessions

OpenClaw had accumulated 48 active sessions, mostly from WhatsApp groups. The cleanup command only clears metadata. The actual transcript files — JSONL logs — survived on disk and were reloaded on restart.

A 2.1MB Transcript Nobody Knew About

The main session's transcript had grown to 2.1 megabytes. Nine hundred lines of conversation history, sent in full with every single request.



Detailed token breakdown

The Wrong Pricing Model

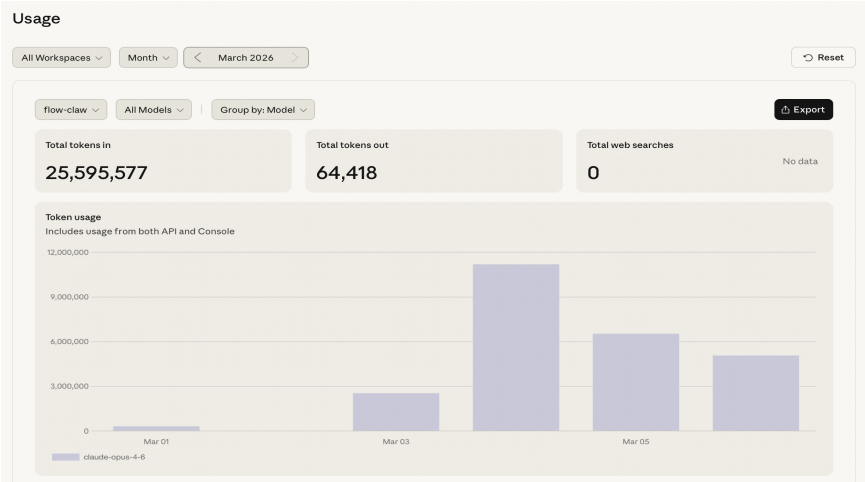
Claude Sonnet 4.5 charges $3.75 per million tokens for prompt cache writes, on top of $3 per million for regular input. Gemini 2.5 Pro charges a flat $1.25 per million input tokens. No separate caching surcharges.

What We Refused to Sacrifice

- Eva's identity (SOUL.md had to remain intact)
- Security posture (Docker isolation, allowlists, manual confirmation)
- Operational capability
- Cost sustainability

What We Changed

- Locked down WhatsApp (dmPolicy to "disabled", groupPolicy to "allowlist")
- Extended cache lifetime (TTL from 5 min to 1 hour)
- Killed 48 zombie sessions and their JSONL transcript files
- Trimmed the system prompt (~3,750 tokens saved)
- Switched to Gemini 2.5 Pro (flat pricing)
- Added Telegram sender allowlist (0 security criticals)



After optimization: 14K tokens per request

The Numbers

Metric	Before	After
Input tokens/request	169,000	14,000
Cost per message	$0.63	$0.02
Active zombie sessions	48	0
Security criticals	1	0
System prompt tokens	~35,000	~14,000

What This Teaches

8 lessons about measure tokens before features, open inputs = open cost surfaces, session cleanup, dynamic prompts, silent config failures, pricing structure, workspace files, and security=cost.

Why We Did This

Eva isn't a product we deployed — she's an experiment we ran for the community. We knew OpenClaw would have rough edges. Eva is lean now. 14K tokens per request. $0.02 per message. Zero security criticals.

— Eliran Keren, Founder of Deeplica